

Informe: Función Hash de Una Sola Vía

Evolución, Implementación Inspirada en SHA-256
y Resolución de Colisiones con Sondeo Lineal

Luis Fernando Lopez Pardo – 20241020045

Sofía Rivera – 20242020224

Juan Esteban Moreno Durán – 20232020107

Juan Esteban Avila Trujillo – 20251020054

Facultad de Ingeniería

Universidad Distrital Francisco José de Caldas

Asignatura:

Ciencias de la Computación II



**UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS**

25 de junio de 2026

Índice

1	Introducción	3
2	Funcionamiento General del Algoritmo Original	3
3	Fórmulas Utilizadas en la Versión Original	3
3.1	Reducción del código ASCII (pérdida de información)	3
3.2	Actualización del estado interno	4
3.3	Rotación de bits	4
3.4	Mezcla adicional (finalización por carácter)	4
4	¿Por Qué es una Función de Una Sola Vía?	4
5	Uso de la Sal (Salt)	5
6	Generador de Números Pseudoaleatorios	5
6.1	¿Por qué no se integró dentro de la función hash?	5
7	¿Cada Hash Generado es Único?	6
7.1	Consistencia	6
7.2	Unicidad y colisiones	6
8	Evolución del Algoritmo: Versión Inspirada en SHA-256	6
8.1	Arquitectura de cuatro bloques (256 bits)	6
8.2	Uso del valor ASCII directo	7
8.3	Fórmulas de la Ronda 1	7
8.4	Mezcla cruzada entre bloques (efecto avalancha)	7
8.5	Ronda de finalización	8
8.6	Salida de longitud fija: 64 caracteres hexadecimales	8
8.7	Comparación con SHA-256 real	8
9	Tablas Hash y Colisiones	8
9.1	Concepto de tabla hash	8
9.2	Tipos de colisión	9
9.3	Técnicas de resolución de colisiones	9
10	Sondeo Lineal	9
10.1	Funcionamiento	9
10.2	Implementación en Java	9
10.3	Problema del agrupamiento (Clustering)	10
11	Demostración de Colisiones con Hash Reducido	10
11.1	¿Por qué reducir el hash?	10
11.2	Probabilidad de colisión (Paradoja del Cumpleaños)	10
11.3	Conversión de hash a índice de tabla	11
11.4	Estructura del programa completo	11
11.5	Código completo de la Parte 2	11

12 Conclusiones

12

1 Introducción

En este informe se documenta el diseño, evolución e implementación de una función hash de una sola vía. El objetivo principal fue construir un algoritmo que transforme una palabra en un código de forma que sea sencillo obtener el hash a partir de la entrada, pero prácticamente imposible recuperar la entrada original a partir del hash.

El desarrollo del algoritmo atravesó varias versiones. Las versiones iniciales exploraron conceptos básicos de mezcla de bits y reducción de información. Posteriormente se construyó una versión más robusta inspirada en el estándar **SHA-256**, que produce una salida de **256 bits fijos** mediante cuatro bloques de estado y mezcla cruzada entre ellos.

Finalmente, se amplió el programa para incluir una demostración de **colisiones hash** usando una versión reducida del hash (1 carácter hexadecimal), resuelta mediante la técnica de **Sondeo Lineal**.

2 Funcionamiento General del Algoritmo Original

El algoritmo recibe dos entradas: una palabra ingresada por el usuario y una sal (*salt*) fija definida internamente. Ambas se concatenan para formar la cadena que será procesada.

Los pasos generales del algoritmo original son:

1. Concatenar la palabra con la sal: $\text{Entrada} = \text{Palabra} + \text{Salt}$.
2. Obtener el código ASCII de cada carácter de la cadena.
3. Reducir cada código ASCII sumando sus dígitos (pérdida de información).
4. Mezclar el resultado con un estado interno acumulado.
5. Aplicar rotación de bits y operación XOR para dispersar los valores.
6. Producir el hash final a partir del estado resultante.

3 Fórmulas Utilizadas en la Versión Original

3.1 Reducción del código ASCII (pérdida de información)

Cada carácter de la cadena se convierte primero a su valor ASCII y luego se reemplaza por la suma de sus dígitos decimales. La fórmula general es:

$$S(A) = d_1 + d_2 + \dots + d_n$$

donde A es el valor ASCII del carácter y $d_1, d_2, \dots, d_n$ son los dígitos que lo conforman.

Ejemplo:

Carácter	ASCII	Suma de dígitos
'A'	65	$6 + 5 = 11$
'8'	56	$5 + 6 = 11$
'J'	74	$7 + 4 = 11$

Los tres caracteres distintos producen el mismo valor reducido (11), lo que confirma la pérdida irreversible de información.

3.2 Actualización del estado interno

Después de obtener $S(A)$ para cada carácter, el estado interno se actualiza mediante una multiplicación seguida de una operación XOR:

$$\text{Estado} = (\text{Estado} \times 131) \oplus S(A)$$

donde 131 es una constante prima elegida para dispersar los bits del estado y $\oplus$ denota la operación XOR a nivel de bits.

3.3 Rotación de bits

Tras la actualización, se realiza una rotación circular de 5 posiciones hacia la izquierda sobre el estado:

$$\text{Estado} = \text{RotL}(\text{Estado}, 5)$$

Esta operación redistribuye los bits del estado, haciendo que cualquier pequeño cambio en la entrada genere diferencias amplias en el hash final (efecto avalancha).

3.4 Mezcla adicional (finalización por carácter)

Al terminar cada carácter se aplica una última mezcla con un desplazamiento de bits:

$$\text{Estado} = \text{Estado} \oplus (\text{Estado} \gg 7)$$

El desplazamiento a la derecha de 7 posiciones y la posterior operación XOR garantizan que los bits de mayor peso también influyan en los bits de menor peso del estado.

4 ¿Por Qué es una Función de Una Sola Vía?

La propiedad de irreversibilidad del algoritmo se fundamenta principalmente en la pérdida de información generada en la etapa de reducción de dígitos.

Como se mostró anteriormente, múltiples valores ASCII distintos pueden producir el mismo valor reducido:

$$65 \rightarrow 11, \quad 56 \rightarrow 11, \quad 74 \rightarrow 11$$

Si solo se conoce el valor 11, es imposible determinar con certeza cuál era el código ASCII original. Esta ambigüedad se propaga a través de todas las demás operaciones.

Adicionalmente:

- No existe una operación inversa directa para deshacer el XOR con un estado desconocido.
- La rotación de bits no es invertible sin conocer el estado exacto en ese punto del proceso.
- El encadenamiento de operaciones hace que un pequeño cambio en la entrada modifique drásticamente el hash final.

5 Uso de la Sal (Salt)

Además de la palabra ingresada por el usuario, el algoritmo incorpora una sal fija. La entrada real que procesa el algoritmo es:

$$\text{Entrada} = \text{Palabra} + \text{Salt}$$

Por ejemplo, si la palabra es HOLA y la sal es Z9xY:

$$\text{Entrada} = \text{HOLA} + \text{Z9xY} = \text{HOLAZ9xY}$$

La sal cumple dos funciones importantes:

- Hace que el hash dependa no solo de la palabra sino también de un valor adicional controlado por el diseñador del sistema.
- Dificulta los ataques de diccionario precalculados (*rainbow tables*), ya que un atacante necesitaría conocer la sal para intentar reproducir el hash.

6 Generador de Números Pseudoaleatorios

6.1 ¿Por qué no se integró dentro de la función hash?

Se tomó la decisión de no incorporar un generador de números pseudoaleatorios dentro de la función hash. La razón principal es que una función hash debe ser **determinista**: la misma entrada siempre debe producir exactamente el mismo hash.

Si se incorporaran números pseudoaleatorios directamente en el proceso, el resultado cambiaría entre distintas llamadas al algoritmo, incluso para la misma palabra de entrada:

$$\begin{aligned}\text{Hash}(\text{HOLA}) &= 12345 && \text{(primera ejecución)} \\ \text{Hash}(\text{HOLA}) &= 67890 && \text{(segunda ejecución)}\end{aligned}$$

Esto inutilizaría el propósito del hash. El generador pseudoaleatorio sí podría utilizarse para generar la sal de forma dinámica antes de invocar la función hash, o para derivar una clave de sesión, pero no dentro del núcleo del algoritmo.

7 ¿Cada Hash Generado es Único?

7.1 Consistencia

El algoritmo es completamente determinista: la misma entrada siempre produce el mismo resultado.

7.2 Unicidad y colisiones

Que el resultado sea consistente no implica que sea único para todas las entradas posibles. El fenómeno en el que dos entradas distintas producen el mismo hash se denomina **colisión**.

Las colisiones son inevitables en cualquier función hash por el **Principio del Palomar**: el dominio de entrada es infinito, mientras que el rango de salida es finito, por lo tanto necesariamente deben existir colisiones.

Definición formal de colisión hash: Se produce una colisión cuando dos entradas distintas $x \neq y$ generan exactamente el mismo valor de salida:

$$H(x) = H(y), \quad x \neq y$$

8 Evolución del Algoritmo: Versión Inspirada en SHA-256

La versión original del algoritmo presentaba dos limitaciones importantes que motivaron su evolución:

1. **Salida de longitud variable:** el resultado numérico podía tener entre 1 y 19 dígitos según la palabra ingresada.
2. **Mayor probabilidad de colisión:** la reducción de dígitos ASCII agrupaba muchos caracteres bajo el mismo valor, aumentando las colisiones.

La nueva versión resuelve ambas limitaciones inspirándose en el estándar **SHA-256**.

8.1 Arquitectura de cuatro bloques (256 bits)

En lugar de un único estado, la versión mejorada utiliza **cuatro bloques de 64 bits**, sumando un total de 256 bits de estado interno:

Bloques	Bits por bloque	Total
4	64 bits	256 bits

Los valores iniciales de cada bloque se toman directamente de SHA-256 (partes fraccionarias de las raíces cuadradas de los primeros cuatro números primos):

```

1 long[] estado = {
2     0x6a09e667f3bcc908L, // bloque 0 - raiz(2)
3     0xbb67ae8584caa73bL, // bloque 1 - raiz(3)
4     0x3c6ef372fe94f82bL, // bloque 2 - raiz(5)
5     0xa54ff53a5f1d36f1L  // bloque 3 - raiz(7)
6 };

```

8.2 Uso del valor ASCII directo

A diferencia de la versión original, esta versión utiliza el valor ASCII completo sin reducción de dígitos, eliminando la fuente artificial de colisiones:

$$\text{ascii} = \text{entrada.charAt}(i) \quad (\text{sin suma de dígitos})$$

8.3 Fórmulas de la Ronda 1

Por cada carácter procesado, se aplican cuatro operaciones sobre el bloque $b = i \bmod 4$:

$$\text{estado}[b] = (\text{estado}[b] \times 131) \oplus \text{ascii} \quad (1)$$

$$\text{estado}[b] = \text{RotL}(\text{estado}[b], 5) \quad (2)$$

$$\text{estado}[b] = \text{estado}[b] \oplus (\text{estado}[b] \gg 7) \quad (3)$$

$$\text{estado}[b] = \text{estado}[b] \oplus C_b \quad (4)$$

donde C_b son las constantes de ronda basadas en raíces cuadradas de números primos, tomadas del estándar SHA-256:

Bloque	Constante	Primo base
0	0x428a2f98d728ae22	2
1	0x7137449123ef65cd	3
2	0xb5c0fbcfec4d3b2f	5
3	0xe9b5dba58189dbbc	7

8.4 Mezcla cruzada entre bloques (efecto avalancha)

Tras actualizar el bloque b , se aplica una **mezcla cruzada** que propaga los cambios a los tres bloques restantes. Esto garantiza que un cambio en un solo carácter afecte los cuatro bloques simultáneamente:

$$\text{estado}[(b+1) \bmod 4] \oplus= \text{RotL}(\text{estado}[b], 13) \quad (5)$$

$$\text{estado}[(b+2) \bmod 4] \oplus= (\text{estado}[b] \gg 17) \quad (6)$$

$$\text{estado}[(b+3) \bmod 4] += \text{estado}[b] \quad (7)$$

8.5 Ronda de finalización

Se aplican cuatro pasadas adicionales de mezcla global para garantizar el efecto avalancha completo, especialmente en entradas cortas:

$$\text{estado}[b] = \text{RotL}(\text{estado}[b], 7) \quad (8)$$

$$\text{estado}[b] = \text{estado}[b] \oplus \text{estado}[(b + 1) \bmod 4] \quad (9)$$

$$\text{estado}[b] = \text{estado}[b] \oplus C_b \quad (10)$$

$$\text{estado}[b] = \text{estado}[b] \oplus (\text{estado}[b] \ggg 13) \quad (11)$$

8.6 Salida de longitud fija: 64 caracteres hexadecimales

La salida siempre es exactamente **64 caracteres hexadecimales = 256 bits**, sin importar la longitud de la entrada. Esto se logra con el formato `%016x` de Java:

```
1 return String.format("%016x%016x%016x%016x",
2     estado[0], estado[1], estado[2], estado[3]);
3 // 16 chars + 16 chars + 16 chars + 16 chars = 64 chars = 256
   bits
```

El especificador `%016x` significa: hexadecimal, mínimo 16 caracteres, relleno con ceros a la izquierda. Esto garantiza que incluso si el valor numérico es pequeño, la salida siempre ocupe exactamente 16 posiciones.

Entrada	Longitud de salida	Bits
"A"	64 chars	256
"HOLA"	64 chars	256
"PROGRAMACION"	64 chars	256

8.7 Comparación con SHA-256 real

Propiedad	SHA-256 real	Nuestra versión
Salida de longitud fija	✓	✓
256 bits de salida	✓	✓
Resistencia a colisiones probada matemáticamente	✓	×
Efecto avalancha completo garantizado	✓	Parcial
Constantes de ronda basadas en primos	✓	✓
Uso en producción	✓	× (educativo)

9 Tablas Hash y Colisiones

9.1 Concepto de tabla hash

Una tabla hash es una estructura de datos que utiliza el hash de un elemento como **índice directo** en un arreglo, permitiendo acceso en tiempo $O(1)$:

$$\text{índice} = H(\text{palabra}) \bmod \text{TAMAÑO_TABLA}$$

9.2 Tipos de colisión

Colisión hash (criptográfica): dos entradas distintas producen exactamente el mismo valor de hash:

$$H(x) = H(y), \quad x \neq y$$

Con 256 bits de salida y un diccionario de 9 palabras, la probabilidad de colisión es:

$$P \approx \frac{1}{2^{256}} \approx 0$$

prácticamente imposible de observar en la práctica.

9.3 Técnicas de resolución de colisiones

Existen dos familias principales de técnicas:

Técnica	Familia	Descripción
Encadenamiento	Hashing Abierto	Lista enlazada por posición
Sondeo Lineal	Hashing Cerrado	Busca siguiente posición libre
Sondeo Cuadrático	Hashing Cerrado	Salta posiciones en $1^2, 2^2, 3^2 \dots$
Doble Hashing	Hashing Cerrado	Aplica segundo hash para saltar

10 Sondeo Lineal

El **Sondeo Lineal** (*Linear Probing*) es la técnica de resolución de colisiones implementada en este proyecto. Cuando una posición está ocupada, avanza una posición a la vez hasta encontrar un espacio libre.

10.1 Funcionamiento

Dado un hash que apunta a la posición i de una tabla de tamaño m , si $\text{tabla}[i]$ está ocupado, se prueba:

$$i_k = (i + k) \bmod m, \quad k = 1, 2, 3, \dots$$

hasta encontrar $\text{tabla}[i_k] = \text{null}$.

10.2 Implementación en Java

```

1 while (tabla[indice] != null) {
2     // Colision detectada - mismos hash
3     if (tablaHashes[indice].equals(hashReducido)) {
4         System.out.println("COLISION: " + palabra +

```

```

5         " y " + tabla[indice] +
6         " generan el mismo hash: " +
        hashReducido);
7     }
8     // Sondeo: avanzar a siguiente posicion
9     indice = (indice + 1) % TAMANIO_TABLA;
10    pasos++;
11 }
12 // Posicion libre encontrada
13 tabla[indice] = palabra;
14 tablaHashes[indice] = hashReducido;

```

El operador % TAMANÍO_TABLA hace que la tabla sea **circular**: al llegar al último índice, vuelve al inicio.

10.3 Problema del agrupamiento (Clustering)

El sondeo lineal tiene una desventaja conocida: cuando varias palabras caen en posiciones contiguas se forma un **bloque** que crece y hace más lentas las inserciones posteriores:

Posición	Contenido
5	HOLA ← bloque
6	JAVA ← bloque
7	ADMIN ← bloque
8	(vacío) ← nueva inserción tiene que saltar 3 posiciones

Este fenómeno se denomina *clustering primario* y es la principal desventaja del sondeo lineal frente al sondeo cuadrático o el doble hashing.

11 Demostración de Colisiones con Hash Reducido

11.1 ¿Por qué reducir el hash?

Con 256 bits de salida y solo 9 palabras en el diccionario, la probabilidad de observar una colisión es prácticamente nula. Para demostrar el fenómeno en la práctica, se reduce el hash a **1 carácter hexadecimal** (los primeros 4 bits de la salida de 256 bits):

$$H_{\text{reducido}}(\text{palabra}) = H_{256}(\text{palabra}).\text{substring}(0, 1)$$

Esto produce exactamente **16 valores posibles** (de 0 a F).

11.2 Probabilidad de colisión (Paradoja del Cumpleaños)

Con $n = 9$ palabras en un espacio de $m = 16$ posibles hashes, la probabilidad de que al menos dos palabras colisionen es:

$$P(\text{colisión}) = 1 - \frac{16!}{(16-9)! \cdot 16^9} = 1 - \frac{16 \times 15 \times \dots \times 8}{16^9} \approx 94\%$$

Con 9 palabras en 16 posibles valores, las colisiones son casi seguras.

11.3 Conversión de hash a índice de tabla

El carácter hexadecimal se convierte directamente a su valor numérico (0–15) como índice:

```

1 public static int hashAIndice(String hashReducido) {
2     return Integer.parseInt(hashReducido, 16); // "a" -> 10, "f"
3     -> 15
4 }

```

Con tamaño de tabla igual a 16, el índice coincide exactamente con el hash reducido, sin necesidad de aplicar módulo adicional.

11.4 Estructura del programa completo

El programa final integra dos partes en un mismo archivo:

	Parte 1	Parte 2
Bits de hash	256 bits (64 chars hex)	4 bits (1 char hex)
Valores posibles	2^{256}	16
Colisiones con 9 palabras	Prácticamente 0	$\approx 94\%$
Resolución	No necesaria	Sondeo lineal
Tamaño de tabla	No aplica	16 posiciones

11.5 Código completo de la Parte 2

```

1 // 1 char hex = 4 bits = 16 valores posibles (0 a F)
2 private static final int TAMANIO_TABLA = 16;
3 private static String[] tabla = new String[TAMANIO_TABLA];
4 private static String[] tablaHashes = new String[TAMANIO_TABLA];
5
6 public static String generarHashReducido(String palabra, String
7     sal) {
8     return generarHash(palabra, sal).substring(0, 1);
9 }
10
11 public static void insertar(String palabra, String sal) {
12     String hashReducido = generarHashReducido(palabra, sal);
13     int indiceOriginal = hashAIndice(hashReducido);
14     int indice = indiceOriginal;
15     int pasos = 0;

```

```
15
16 while (tabla[indice] != null) {
17     if (tablaHashes[indice].equals(hashReducido)) {
18         // Colision real: mismo hash, palabra diferente
19         System.out.println("COLISION: " + palabra + " y " +
20             tabla[indice] + " -> hash: " + hashReducido)
21     ;
22         System.out.println("Aplicando sondeo lineal...");
23     }
24     indice = (indice + 1) % TAMANIO_TABLA;
25     pasos++;
26     if (pasos == TAMANIO_TABLA) {
27         System.out.println("Tabla llena.");
28         return;
29     }
30     tabla[indice] = palabra;
31     tablaHashes[indice] = hashReducido;
32     System.out.println("Insertado en posicion " + indice +
33         " (pasos: " + pasos + ")");
34 }
```

12 Conclusiones

Se diseñó, implementó y evolucionó una función hash de una sola vía en tres etapas progresivas, culminando en una versión inspirada en SHA-256 con resolución de colisiones.

Los aspectos más relevantes del trabajo son:

1. **Irreversibilidad:** la propiedad de una sola vía de la versión original se logra mediante la pérdida de información en la suma de dígitos. La versión mejorada mantiene esta propiedad a través de operaciones XOR encadenadas, rotaciones y mezcla cruzada.
2. **Salida de longitud fija:** la versión inspirada en SHA-256 produce siempre exactamente 64 caracteres hexadecimales (256 bits), independientemente de la longitud de la entrada, gracias al especificador de formato `%016x` aplicado a los cuatro bloques de estado.
3. **Espacio de salida y colisiones:** con 256 bits la probabilidad de colisión con un diccionario pequeño es prácticamente nula. Al reducir a 1 carácter hexadecimal (16 valores posibles), la probabilidad de colisión con 9 palabras asciende a aproximadamente el 94%, demostrando empíricamente el fenómeno.
4. **Sondeo Lineal:** la técnica implementada detecta colisiones (mismo hash, palabras distintas) y las resuelve avanzando linealmente hasta encontrar una posición libre en la tabla. Su principal limitación es el agrupamiento (*clustering*) que puede degradar el rendimiento de $O(1)$ a $O(n)$ en el peor caso.
5. **Uso de sal:** la incorporación de una sal fija fortalece el algoritmo al hacer

que el hash dependa de un valor adicional, dificultando ataques de diccionario precalculados.